# ANTS: Adversarial Agentic Task Synthesis

Team project with Daniel Totolici · Submitted to the Meta Global AI Hackathon

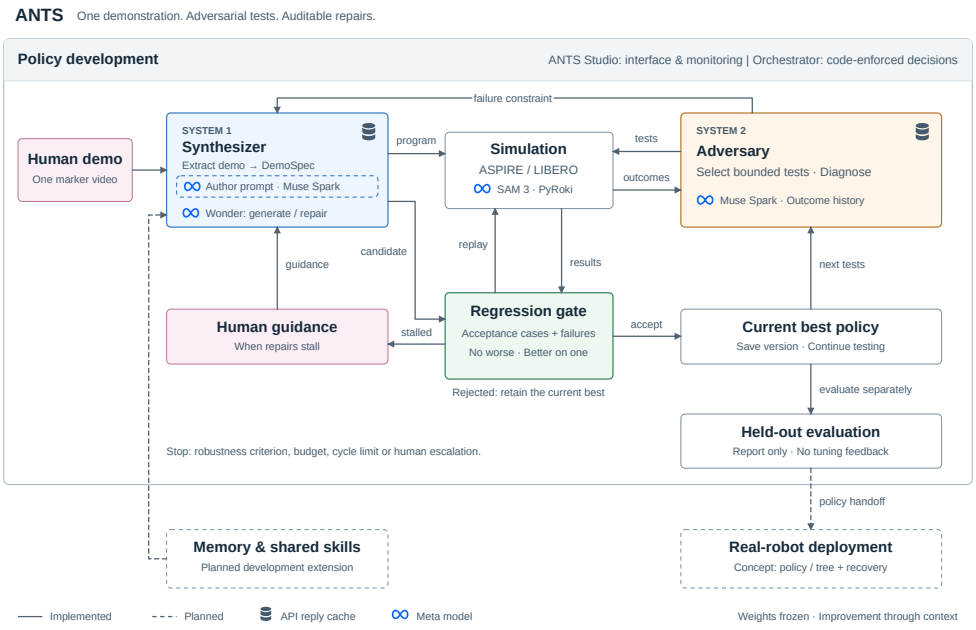
Code as policy | Muse Wonder | Muse Spark | SAM 3/3.1 | LIBERO-Pro

## One Demo In, a Hardened Robot Program Out

SUBMITTED META GLOBAL AI HACKATHON

Show a robot a task **once on video**. A frozen model writes the robot program. A second system attacks it until the edge cases are covered. **Nothing is trained.** The skill improves through context only.

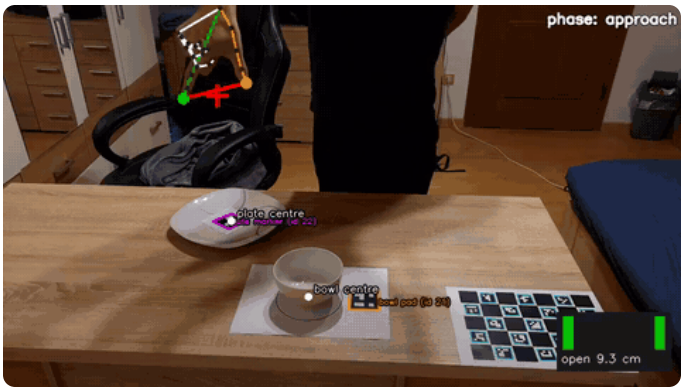
**Submitted to the Meta Global AI Hackathon**, theme *Push the Frontier*: use Meta's models to tackle challenges in robotics and beyond. ANTS pushes it with Meta's own models in both systems: Muse Wonder writes the robot program, Muse Spark plans and diagnoses the attacks, and SAM 3/3.1 gives the robot its perception.



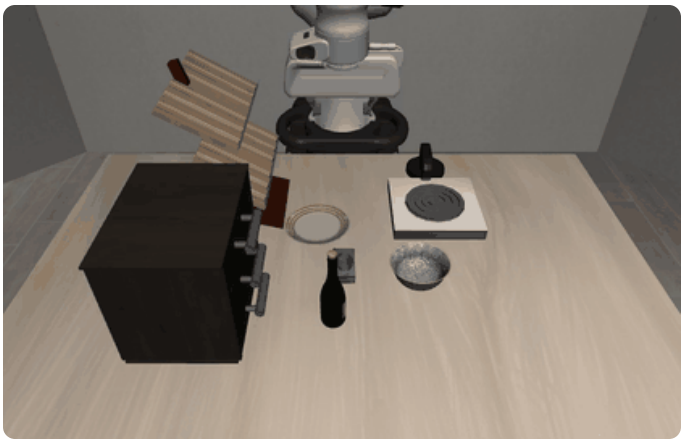

Two systems, one goal: a robust policy. Solid boxes are built, dashed boxes are planned.

## A Human Video as the Prompt

A text prompt says *"put the bowl on the plate"*. It cannot say *where* to grasp, *how high* to carry or *where* to release. A human video can. We track the hand and objects in one demo and turn them into numbers relative to object size: grasp the rim, approach from above, carry height, release over the plate. The VLM writes the robot program from these numbers instead of guessing them.




One human demo, with what the tracker reads.



v0, the first candidate policy generated and tested by System 1, run in LIBERO-Pro.

| Prompt to the generator | Solved | Note |
|---|---|---|
| Text only | 0/10 | 2 samples, 5 acceptance seeds each |
| Text + human video | 5/5 | best of 2 samples, the other sample failed |

This is an early hint, not a result: one task, one clip, two samples per arm. A full demo versus no-demo evaluation is the first item of the outlook. The demo can also come from teleoperation, not only from video.

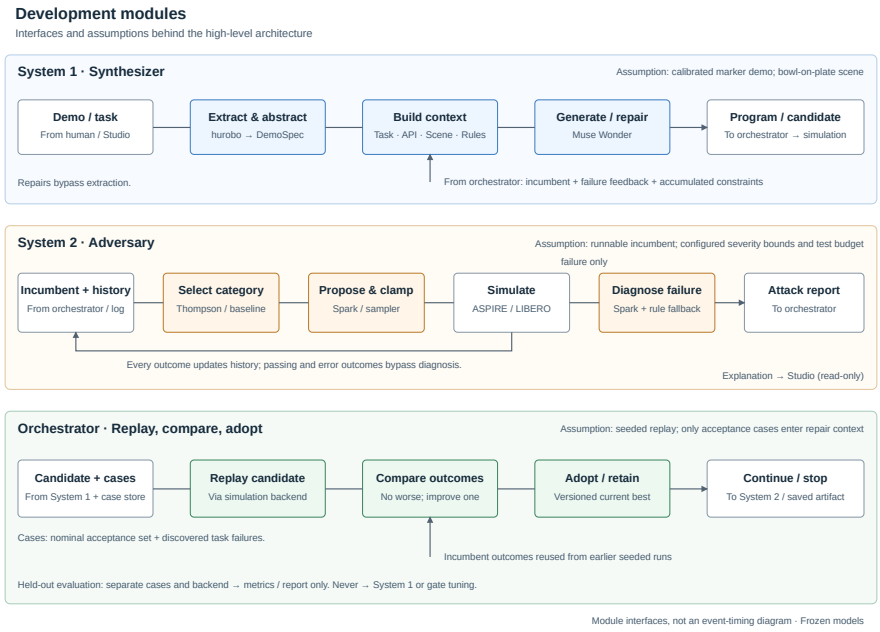
## System 1 and System 2, Working on One Objective

Two systems run independently, talk through a small contract and share one goal: a policy that does not break.

- **System 1, policy generator (synthesizer):** turns the prompt (human video, teleop or text) into a **program as policy**, checks it in simulation and repairs it.
- **System 2, failure case generator (adversary), built by Daniel Totolici:** looks for the cases most likely to break the policy. It picks the weakest category, proposes perturbations within a bounded severity (pose, size, physics, actuation, camera, lighting), runs them and diagnoses each failure into a rule.
- **Gate:** every rule goes back into System 1's context. A rewrite is accepted only if it is no worse on all cases seen so far and better on one.

Each system makes the other better: the generator hardens the program, the adversary keeps finding what the hardening missed. When the loop stalls it asks a human, and the answer becomes a rule.

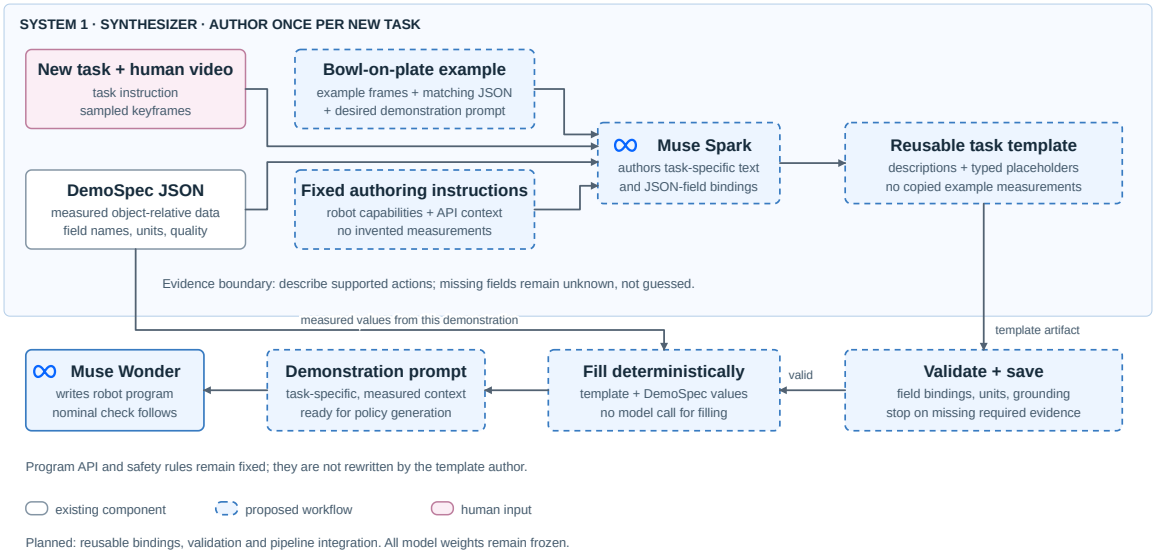
*Note: I worked on the synthesizer and the demo-to-program idea. I did not look into the technical details of the adversary, which is Daniel's part.*




Modules and interfaces of the development loop.

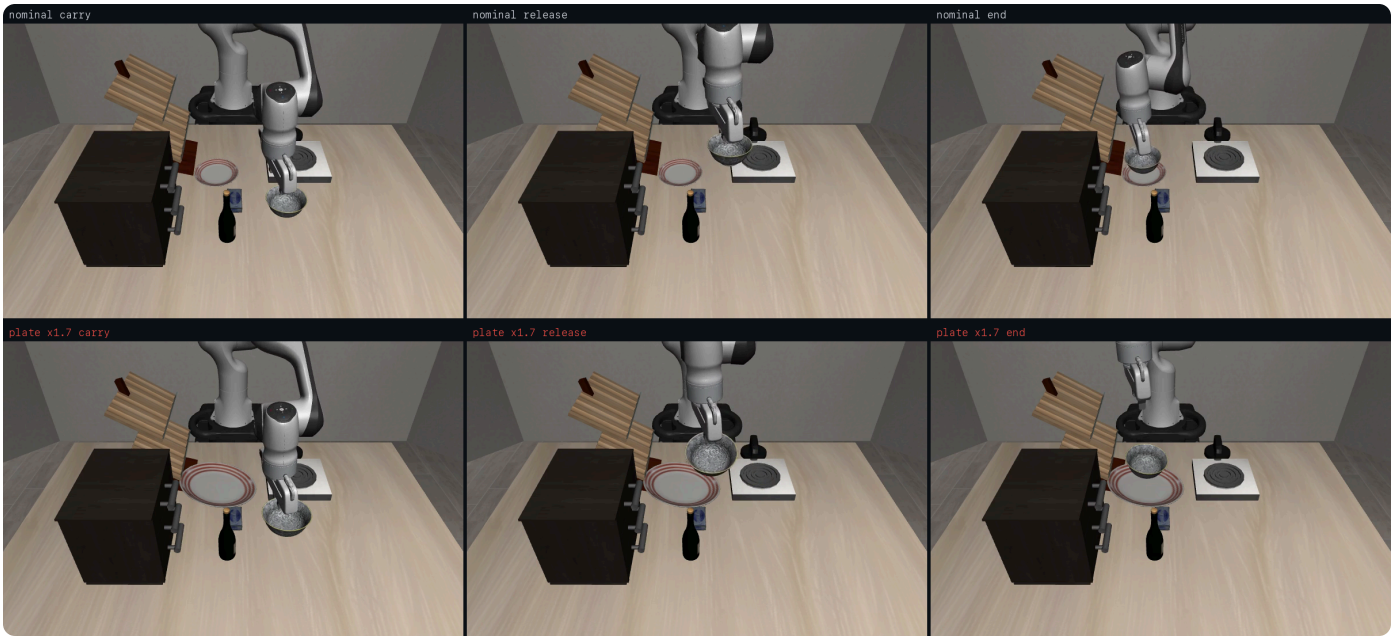
## Author prompt

Proposed reusable-template workflow. The existing prototype produces a filled paragraph instead.




Planned: an author step turns the demo evidence into a reusable task prompt before the program is written.

## What the Adversary Found




Nominal (top) versus plate ×1.7 (bottom), same seed.

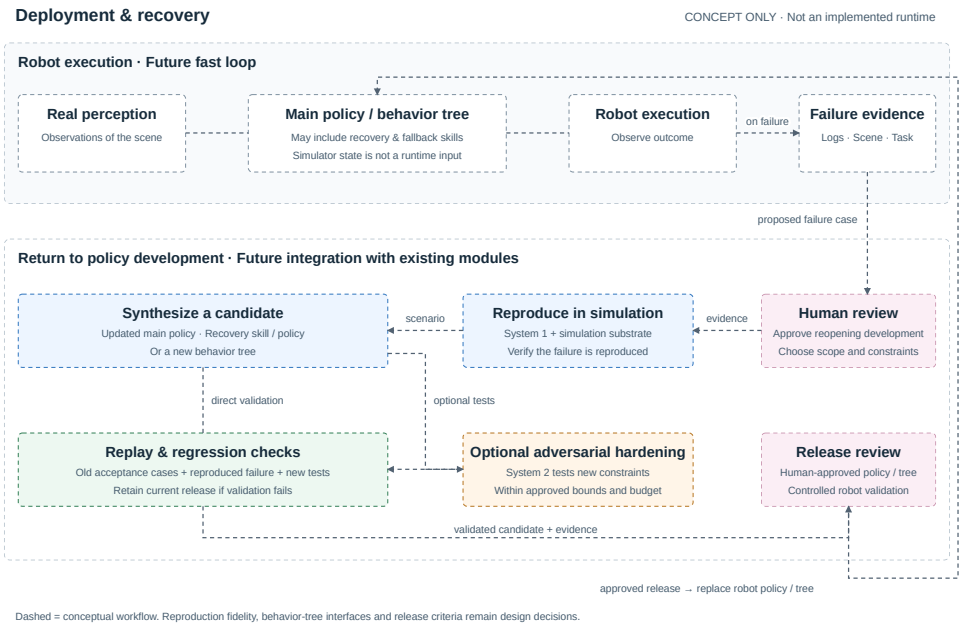
**v0** is the first candidate policy generated and tested by System 1. Each accepted rewrite becomes the next version (v1, v2, ...). With a larger plate, SAM 3 measured the plate 11 cm tall instead of 2 cm. The program released the bowl about 9 cm too high. Muse Spark traced it to one line, `plate_top_z = max(...)`, Muse Wonder rewrote it and the gate accepted the fix.

| Question | Result | From |
|---|---|---|
| One demo gives a working program? | v0 passes 47/50 held-out perturbed cases | 759 LIBERO rollouts |
| Adversary finds a real weak spot? | 0 failures in 599 rollouts with correct perception, 11 in 26 with a mis-read plate | one blind spot |
| Gate blocks bad fixes? | 3 of 3 bad rewrites rejected | logged runs |
| Fix generalises? | **Not yet.** Held-out size cases: v0 49/50, v1 47/50 | gate saw only 4 cases |
| Adaptive beats uniform testing? | **Not answered.** 2/200 vs 1/200 failures | v0 fails too rarely |

Held-out cases use scene seeds the system never tuned on. Cost: about $12 of model tokens for the whole study. Limits are stated, not hidden.

## Add-on: Recovery Policy

A single policy that handles every edge case becomes long, hard to read and hard to reuse. The architecture can instead let System 1 write a **main policy plus small recovery policies**. This works in the development phase and later in deployment: a production failure is reproduced in simulation, the adversary hardens the fix, and the new recovery skill is released after validation. A final step could compose main and recovery skills into a **behavior tree**, which keeps each part modular and reusable. Not built yet.




Deployment and recovery concept (not implemented).

## Outlook

- **Comprehensive evaluation** of demo versus no demo, and a broader adversary, across more tasks and acceptance sets. Current tests are a start, not comprehensive.
- **Better planning:** cuRobo trajectory optimisation against a static 3D map of the scene. A scene reconstruction probe (TSDF to ESDF) already works.
- **Sim-to-real gap:** today the gap is assumed to be zero, so robust in simulation means robust in reality. Next: gap-aware test cases, or a real2sim digital twin that replicates the real scene 1:1.
- **Deployment phase:** today ANTS covers policy development only.
- **ANTS Studio demo: coming soon!** One window to watch the whole loop: the synthesizer chat with live cameras, and the adversary's attacks, replays and versions.
- **Memory:** store hardened policies and share skills across tasks.

## Team

Alfredo: synthesizer and demo prompt. Daniel: adversary. Alfredo Wijaya Hardjoprawiro and Daniel Totolici. Demonstrations come from hurobo, which turns phone videos of a human hand into robot demos.

Alfredo Wijaya Hardjoprawiro — M.Sc. Artificial Intelligence, TH Ingolstadt

alh7017@thi.de alfredowijaya@gmail.com GitHub LinkedIn alfredowh.github.io